Best Model Overview

Reconstruir el ingreso de cada persona encuentra al 82 % de los hogares pobres

Universidad de los Andes

0,78

F1 en Kaggle: Público 0,77 → Privado 0,78 (**2.° lugar**)

0,782

F1 en el 20 % apartado (holdout)

18

submissions en Kaggle (7 algoritmos + mezclas)

El modelo

1. Estimamos el ingreso de cada persona con lo que el test sí trae: ocupación, horas, cotización.
2. Lo sumamos por hogar y lo comparamos con la línea.
3. Una mezcla liderada por LightGBM decide: pobre si $P \geq 0{,}36$.

Lo que encontramos

1. Saber cuánto gana cada persona vale más que el algoritmo.
2. Ninguna de nuestras alternativas pasa de 0,78: un tercio de los pobres está casi en la línea.
3. Los excluidos están cerca de la línea.

Kaggle: tablas pública y privada (final). LightGBM pesa 76 % de una mezcla con dos redes y un rediseño. Fuente: views/tablas/nivel7/etapa18_resumen.csv.

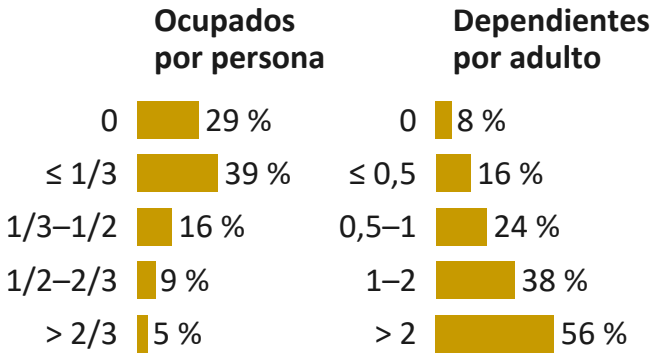


Qué decir (1 min). El gobierno quiere encontrar a los hogares pobres, pero la pobreza se define por el ingreso, y el ingreso es justo lo que no se puede preguntar bien. Nuestra respuesta fue reconstruirlo: estimamos cuánto gana cada persona con preguntas sencillas sobre su trabajo, lo sumamos por hogar y lo comparamos con la línea. Con eso quedamos segundos en Kaggle, con un F1 de 0,77 en la tabla pública y 0,78 en la privada, que es la final. Subimos del público al privado; los equipos en 0,81 en el público bajaron. Elegimos con holdout, no con el leaderboard público. Tres mensajes que vamos a sostener: primero, las buenas variables pesaron mucho más que el algoritmo; segundo, hay un techo cerca de 0,78 porque muchos pobres están casi en la línea; y tercero, los hogares que el modelo deja por fuera están cerca de la línea, no en la pobreza extrema.

Pregunta del profesor: ¿por qué una mezcla y no un solo modelo? Respuesta: La mezcla gana menos de una milésima frente al LightGBM solo, y esa ventaja no sobrevive a una prueba más estricta. La subimos porque cumplió la regla que fijamos antes, pero a una agencia le recomendaríamos el LightGBM solo.

La pobreza cambia por saltos y combinaciones: terreno de los árboles

Universidad de los Andes



Un modelo lineal suma efectos; un árbol corta y combina.

- **Saltos:** con más ocupados, la pobreza sube (29 % a 39 %) y luego cae (5 %).
- **Combinaciones:** el arriendo pesa distinto si la casa es propia.
- 1 de cada 5 hogares es pobre.

Esperamos que los árboles ganen.

Tasa de pobreza en los 164.960 hogares de train; promedio: 20 %. Fuente: scripts/13_figuras_presentacion.R; views/tablas/nivel7/etapa15_ebm_importancia.csv.



Qué decir (1 min). Queremos encontrar hogares pobres sin ver su ingreso, así que lo reconstruimos con lo que sí se ve. Antes de comparar algoritmos, miremos qué forma tiene esa relación. No es una línea recta: cuando nadie en el hogar trabaja, la pobreza es menor que cuando trabaja un tercio, porque hay pensionados. Y las variables se combinan: pagar arriendo no significa lo mismo si la casa es propia o arrendada. Un modelo lineal suma efectos; un árbol corta en umbrales y combina variables solo. Por eso esperábamos que ganaran los árboles. Además, solo uno de cada cinco hogares es pobre, y eso obliga a cuidar el umbral.

Pregunta del profesor: ¿no bastaba con poner interacciones en el Logit? Respuesta: Se puede, pero habría que escogerlas a mano entre más de mil pares posibles; los árboles las encuentran solos. El precio es que se interpretan peor.

Cancha nivelada: mismas 48 variables, misma preparación y misma validación

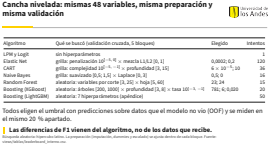
Universidad de los Andes

Algoritmo	Qué se buscó (validación cruzada, 5 bloques)	Elegido	Intentos
LPM y Logit	sin hiperparámetros		1
Elastic Net	grilla: penalización $10^{[-5,\,0]}$ × mezcla L1/L2 [0, 1]	0,0002; 0,2	120
CART	grilla: complejidad $10^{[-5,\,-1]}$ × profundidad [3, 15]	6×10^{-5}; 10	36
Naive Bayes	grilla: suavizado [0,5; 1,5] × Laplace [0, 3]	0,5; 0	16
Random Forest	aleatoria: variables por corte [3, 25] × hoja [5, 60]	23; 24	15
Boosting (XGBoost)	aleatoria: árboles [200, 1000] × profundidad [3, 8] × tasa $10^{[-3,\,-1]}$	781; 6; 0,020	20
Boosting (LightGBM)	aleatoria: 7 hiperparámetros (apéndice)		50

Todos eligen el umbral con predicciones sobre datos que el modelo no vio (OOF) y se miden en el mismo 20 % apartado.

Las diferencias de F1 vienen del algoritmo, no de los datos que recibe.

Búsqueda aleatoria: hipercubo latino. La preparación (imputación, dummies y escalado) se ajusta dentro de cada bloque. Fuente: views/tablas/leaderboard_interno.csv.



Cancha nivelada: mismas 48 variables, misma preparación y misma validación

Universidad de los Andes

Algoritmo	Qué se buscó (validación cruzada, 5 bloques)	Elegido	Intentos
LPM y Logit	sin hiperparámetros	—	1
Elastic Net	grilla: penalización $10^{-5,-4}$ × mezcla L1/L2 [0, 1]	0,00032; 0,2	120
CART	grilla: complejidad $10^{-6,-3}$ × profundidad [3, 15]	4×10^{-5}; 10	36
Naive Bayes	grilla: suavizado [0,5; 1,5] × Laplace [0, 3]	0,5; 0	16
Random Forest	aleatoria: variables por corte [3, 23] × hoja [5, 60]	23; 14	15
Boosting (XGBoost)	aleatoria: árboles [200, 1000] × profundidad [3, 8] × tasa $10^{-1,-0}$	766; 4; 0,020	20
Boosting (LightGBM)	aleatoria: 7 hiperparámetros (ajenidicio)		50

Todos eligen el umbral con predicciones sobre datos que el modelo no vio (OOF) y se miden en el mismo 20 % apartado.

Las diferencias de F1 vienen del algoritmo, no de los datos que recibe.

Búsqueda aleatoria: hiperparámetros, cuya exploración (cuándo parar, cómo dividir) se ajusta dentro de cada bloque. Fuente: www.habitatclimaticoandino.uniandes.edu

Qué decir (1 min). Para que la comparación sea justa, todos los algoritmos reciben exactamente las mismas 48 variables y la misma preparación de datos. Cada uno elige sus hiperparámetros con validación cruzada en cinco bloques: grilla cuando hay uno o dos hiperparámetros, búsqueda aleatoria cuando hay más. El umbral se elige con predicciones sobre hogares que el modelo no vio, y todos se miden en el mismo 20 % apartado. Así, si uno gana, es por el algoritmo y no por los datos.

Pregunta del profesor: ¿no tienen ventaja los que probaron más combinaciones? Respuesta: Poca: XGBoost con 20 intentos ya llegaba a 0,746, frente a 0,749 de LightGBM con 50. Y Elastic Net, con 120, empata con el Logit sin afinar.

Con las mismas variables, el boosting gana y Naive Bayes queda último

Universidad de los Andes

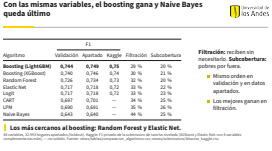
	F1				
Algoritmo	Validación	Apartado	Kaggle	Filtración	Subcobertura
Boosting (LightGBM)	**0,744**	**0,749**	**0,75**	29 %	20 %
Boosting (XGBoost)	0,740	0,746	0,74	30 %	21 %
Random Forest	0,726	0,734	0,73	32 %	20 %
Elastic Net	0,717	0,718	0,72	33 %	22 %
Logit	0,717	0,718	0,72	33 %	23 %
CART	0,697	0,701	—	34 %	25 %
LPM	0,690	0,691	—	35 %	26 %
Naive Bayes	0,643	0,640	—	44 %	25 %

Filtración: reciben sin necesitarlo. **Subcobertura:** pobres por fuera.

- Mismo orden en validación y en datos apartados.
- Los mejores ganan en filtración.

Los más cercanos al boosting: Random Forest y Elastic Net.

48 variables, 32.993 hogares apartados (holdout). Kaggle: F1 privado de la submission de cancha nivelada (XGBoost y Elastic Net: con 8 variables complementarias más); —: no subido. Fuente: views/tablas/comparacion_algoritmos.csv; stores/submissions/bitacora_kaggle.csv.



Qué decir (1 min). Estos son los siete algoritmos del curso con las mismas variables. Como esperábamos, el boosting queda primero y los modelos lineales y Naive Bayes atrás. El orden es el mismo en la validación cruzada y en los datos apartados, así que no es suerte de una partición. Fíjense en las dos últimas columnas: todos dejan por fuera a un 20 a 26 % de los pobres; donde más se separan es en la filtración. Naive Bayes queda último porque supone que las variables son independientes, y aquí están muy relacionadas: ocupados, horas y cotizantes se mueven juntos. Los dos rivales más cercanos son Random Forest y Elastic Net.

Pregunta del profesor: ¿son reales estas diferencias o es ruido? Respuesta: Con 32.993 hogares, la diferencia más pequeña que podemos medir está entre 0,003 y 0,006. Brechas de 0,015 o más son reales; la de XGBoost frente a LightGBM, no.

Random Forest encuentra a los mismos pobres, pero incluye más no pobres

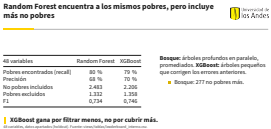
48 variables	Random Forest	XGBoost
Pobres encontrados (recall)	80 %	79 %
Precisión	68 %	70 %
No pobres incluidos	2.483	2.206
Pobres excluidos	1.332	1.358
F1	0,734	0,746

Bosque: árboles profundos en paralelo, promediados. **XGBoost:** árboles pequeños que corrigen los errores anteriores.

- Bosque: 277 no pobres más.

XGBoost gana por filtrar menos, no por cubrir más.

48 variables, datos apartados (holdout). Fuente: views/tablas/leaderboard_interno.csv.



Qué decir (1 min). Random Forest es el rival más cercano. Encuentra prácticamente a los mismos pobres que XGBoost, pero le da el subsidio a 277 hogares no pobres más. ¿Por qué? El bosque construye muchos árboles profundos en paralelo y los promedia; XGBoost construye árboles pequeños uno tras otro, y cada uno se concentra en los errores que quedan, que son los hogares cerca de la línea.

Pregunta del profesor: ¿por qué el bosque incluye más no pobres? Respuesta: Porque promedia árboles que se entrenan por separado y ninguno se especializa en los hogares difíciles; en el boosting, cada árbol nuevo se dedica a corregir esos errores.

Elastic Net empata con el Logit: le faltan cortes, no penalización

Universidad de los Andes

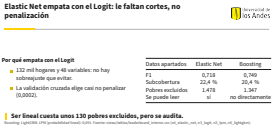
Por qué empata con el Logit

- 132 mil hogares y 48 variables: no hay sobreajuste que evitar.
- La validación cruzada elige casi no penalizar (0,0002).

Datos apartados	Elastic Net	Boosting
F1	0,718	0,749
Subcobertura	22,4 %	20,4 %
Pobres excluidos	1.478	1.347
Se puede leer	sí	no directamente

Ser lineal cuesta unos 130 pobres excluidos, pero se audita.

Boosting: LightGBM. LPM (probabilidad lineal): 0,691. Fuente: views/tablas/leaderboard_interno.csv (n4_elastic_net, n3_logit, n3_lpm, n6_lightgbm).



Qué decir (1 min). El segundo rival es Elastic Net. Su fuerte es penalizar los coeficientes para no sobreajustar, pero con más de 130 mil hogares y solo 48 variables no hay sobreajuste que evitar. La validación cruzada lo confirma: elige una penalización casi nula, y por eso empata con el Logit. Lo que lo separa del boosting no es la regularización sino la forma: una suma lineal no capta umbrales ni combinaciones. En la práctica eso significa unos 130 pobres más excluidos en los datos apartados. A cambio, cada coeficiente se puede leer y auditar.

Pregunta del profesor: ¿y si le agregan interacciones al Elastic Net? Respuesta: Ahí la penalización sí serviría, porque las variables se multiplican. Pero aun así habría que adivinar qué cortes poner; el boosting los encuentra solo.

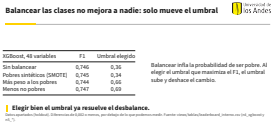
Balancear las clases no mejora a nadie: solo mueve el umbral

XGBoost, 48 variables	F1	Umbral elegido
Sin balancear	0,746	0,36
Pobres sintéticos (SMOTE)	0,745	0,34
Más peso a los pobres	0,744	0,66
Menos no pobres	0,747	0,69

Balancear infla la probabilidad de ser pobre. Al elegir el umbral que maximiza el F1, el umbral sube y deshace el cambio.

Elegir bien el umbral ya resuelve el desbalance.

Datos apartados (holdout). Diferencias de 0,002 o menos, por debajo de lo que podemos medir. Fuente: views/tablas/leaderboard_interno.csv (n4_xgboost y n5_*).

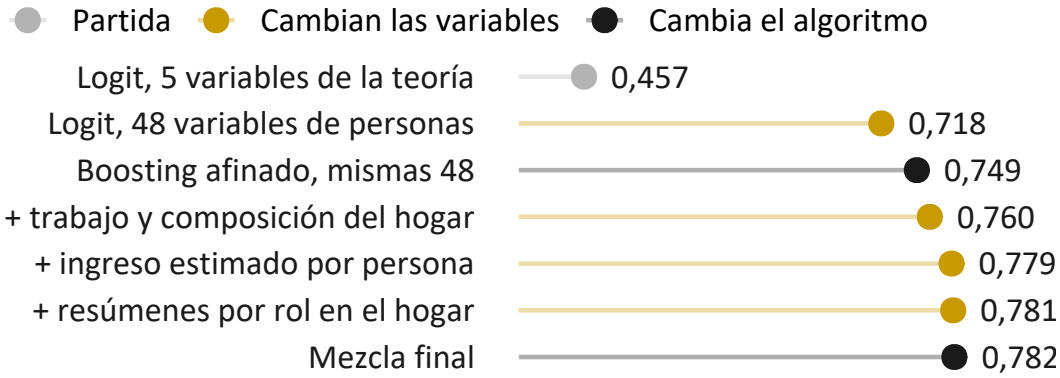


Qué decir (1 min). Solo uno de cada cinco hogares es pobre, así que probamos las tres soluciones clásicas: crear pobres sintéticos, darles más peso o quitar no pobres. Ninguna cambia el F1 en más de dos milésimas. Lo que sí cambia es el umbral, porque balancear solo sube las probabilidades de todos. Si después se elige el umbral que maximiza el F1, ese ajuste ya hace el trabajo.

Pregunta del profesor: ¿y si el umbral fuera fijo en 0,5? Respuesta: Ahí balancear sí ayudaría, porque acerca las probabilidades al corte; no lo medimos. Elegir el umbral es la forma directa de lograr lo mismo.

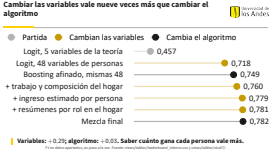
Cambiar las variables vale nueve veces más que cambiar el algoritmo

Universidad de los Andes



Variables: $+0{,}29$; algoritmo: $+0{,}03$. Saber cuánto gana cada persona vale más.

F1 en datos apartados, un paso a la vez. Fuente: views/tablas/leaderboard_interno.csv y views/tablas/nivel7/.



Qué decir (1 min). Pongamos la comparación en perspectiva. Toda la diferencia entre algoritmos que acabamos de ver suma 0,03 de F1. Construir buenas variables, y sobre todo estimar el ingreso de cada persona con su ocupación, sus horas y su cotización, para luego sumarlo por hogar, sumó 0,29. El salto más grande lo dio el mismo Logit con mejores variables.

Pregunta del profesor: ¿no depende del orden de los pasos? Respuesta: Sí, la suma es secuencial. Pero en cualquier orden el contraste es grande: con LightGBM fijo, las variables nuevas suman 0,032, más que toda la distancia entre el Logit y el boosting.

Con buenas variables, todos los algoritmos llegan al mismo techo

Universidad de los Andes

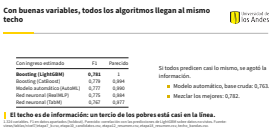
Con ingreso estimado	F1	Parecido
Boosting (LightGBM)	**0,781**	1
Boosting (CatBoost)	0,779	0,994
Modelo automático (AutoML)	0,777	0,990
Red neuronal (RealMLP)	0,775	0,984
Red neuronal (TabM)	0,767	0,977

Si todos predicen casi lo mismo, se agotó la información.

- Modelo automático, base cruda: 0,763.
- Mezclar los mejores: 0,782.

El techo es de información: un tercio de los pobres está casi en la línea.

1.324 variables. F1 en datos apartados (holdout). Parecido: correlación con las predicciones de LightGBM sobre datos no vistos. Fuente: views/tablas/nivel7/etapa7_b.csv, etapa10_candidatos.csv, etapa12_resumen.csv, etapa18_resumen.csv, techo_bandas.csv.



Qué decir (1 min). Fuera de la lista del curso probamos otro boosting, redes neuronales y un modelo automático que prueba decenas de algoritmos solo. Con nuestras variables, todos quedan a menos de 0,015 de LightGBM y predicen casi lo mismo: la correlación es de 0,98 o más. El modelo automático con la base cruda, sin nuestras variables, pierde casi dos puntos. Lo que le faltaba eran variables, no capacidad. El techo viene de que un tercio de los pobres está tan cerca de la línea que en la encuesta se ve igual que un no pobre.

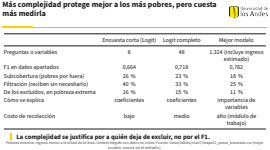
Pregunta del profesor: ¿por qué gana LightGBM y no una red neuronal? Respuesta: En datos de tabla de este tamaño, el boosting suele ganarle a las redes. Aquí la diferencia es de 0,005, apenas por encima de lo que podemos medir.

Más complejidad protege mejor a los más pobres, pero cuesta más medirla

	Encuesta corta (Logit)	Logit completo	Mejor modelo
Preguntas o variables	8	48	1.324 (incluye ingreso estimado)
F1 en datos apartados	0,664	0,718	0,782
Subcobertura (pobres por fuera)	26 %	23 %	18 %
Filtración (reciben sin necesitarlo)	40 %	33 %	25 %
De los excluidos, en pobreza extrema	16 %	15 %	11 %
Cómo se explica	coeficientes	coeficientes	importancia de variables
Costo de recolección	bajo	medio	alto (módulo de trabajo)

La complejidad se justifica por a quién deja de excluir, no por el F1.

Pobreza extrema: ingreso menor a la mitad de la línea. Umbral elegido con datos no vistos. Fuente: views/tablas/nivel7/etapa15_pesos_bienestar.csv (mejor modelo: mezcla sin el rediseño).



Qué decir (1 min). Este es el balance entre complejidad, interpretación y desempeño. Pasar de una encuesta de 8 preguntas a un Logit completo, y luego a nuestro mejor modelo, baja la subcobertura de 26 a 18 % y la filtración de 40 a 25 %. Pero lo más importante es a quién deja por fuera cada uno: con el mejor modelo, solo 11 % de los excluidos está en pobreza extrema; con la encuesta corta, 16 %. El costo es que el mejor modelo necesita preguntar por el trabajo de cada persona, y eso es caro.

Pregunta del profesor: ¿el mejor modelo se puede explicar a un ciudadano? Respuesta: No como un Logit, pero sus decisiones se resumen en dos cosas: ingreso estimado y vivienda. Si la agencia exige transparencia total, un boosting interpretable (EBM) llega a 0,777 con curvas legibles.

¿El modelo con más puntaje en Kaggle? No necesariamente

Universidad de los Andes

1. **Elegir por el costo de cada error.**
2. **LightGBM solo**: rinde igual que la mezcla.
3. **Transparencia**: EBM, boosting interpretable (0,777) o Logit (0,718).
4. **Encuesta corta**: umbral más bajo y verificación en campo.

Decide qué se puede medir y cuánto cuesta excluir.

Fuente: views/tablas/nivel7/etapa15_resumen.csv; views/tablas/leaderboard_interno.csv; views/tablas/nivel7/etapa18_resumen.csv.



Qué decir (1 min). Si una agencia nos pidiera un modelo, no le daríamos necesariamente el de mayor puntaje en Kaggle. La mezcla le gana al LightGBM solo por menos de una milésima, así que recomendamos el más simple, que además es más fácil de mantener. La elección depende de qué se puede medir y de cuánto cuesta excluir, no del puntaje. El F1 supone que excluir y filtrar cuestan igual, y una agencia puede pensar distinto; entonces el umbral se elige por el costo de cada error. Si la agencia necesita explicar cada decisión, un boosting interpretable pierde medio punto. Y si solo puede pagar una encuesta corta, debe compensar con un umbral más bajo y verificación en campo.

Pregunta del profesor: ¿por qué no recomiendan directamente el Logit, que es transparente? Respuesta: Porque deja por fuera a 5 puntos más de pobres, y más de ellos en pobreza extrema. Esa pérdida hay que ponerla en la balanza frente a la transparencia.

Glosario (1/2): cómo medimos los errores

Precisión	De los hogares que el programa atiende, qué parte es pobre.
Filtración	1−precisión: hogares no pobres que reciben el subsidio sin necesitarlo.
Recall	De los hogares pobres, qué parte atiende el programa.
Subcobertura	1−recall: hogares pobres que se quedan por fuera.
F1	Promedio (armónico) de precisión y recall: solo es alto si ambos lo son.
Umbral	Probabilidad desde la que un hogar se declara pobre. Bajarlo cubre más pobres a cambio de más filtración.

Glosario (1/2): cómo medimos los errores

Universidad de los Andes

Precisión	De los hogares que el programa atiende, qué parte es pobre.
Filtración	1 – precisión: hogares no pobres que reciben el subsidio sin necesitarlo.
Recall	De los hogares pobres, qué parte atiende el programa.
Subcobertura	1 – recall: hogares pobres que se quedan por fuera.
F1	Promedio (armónico) de precisión y recall: solo es alto si ambos lo son.
Umbral	Probabilidad desde la que un hogar se declara pobre. Bajarlo cubre más pobres a cambio de más filtración.

Glosario de apoyo; se muestra solo si lo preguntan.

Glosario (2/2): cómo validamos

Datos apartados (holdout)	20 % del train (32.993 hogares) separado al inicio; solo sirve para medir, nunca para decidir.
Validación cruzada	El 80 % restante se parte en 5 bloques: se entrena con 4 y se evalúa en el quinto, rotando.
Predicción sobre datos no vistos (OOF)	La predicción de cada hogar hecha por un modelo que no lo vio. Con ella elegimos umbral, pesos e hiperparámetros.
Modelo de ingreso por persona	Estima cuánto gana cada persona con lo que el test sí trae; sumado al hogar, entra como variable. El que alimenta a un hogar nunca vio ese hogar.
Diferencia más pequeña que podemos medir	Con 32.993 hogares, mejoras menores a $\approx 0{,}0025$ de F1 no se distinguen del ruido.

Glosario (2/2): cómo validamos

Universidad de los Andes

Datos apartados (holdout)	20 % del train (32.993 hogares) separado al inicio; solo sirve para medir, nunca para decidir.
Validación cruzada	El 80 % restante se parte en 5 bloques: se entrena con 4 y se evalúa en el quinto, rotando.
Predicción sobre datos no vistos (OOF)	La predicción de cada hogar hecha por un modelo que no lo vio. Con ella elegimos umbral, pesos e hiperparámetros.
Modelo de ingreso por persona	Estima cuánto gana cada persona con lo que el test sí trae; sumado al hogar, entra como variable. El que alimenta a un hogar nunca vio ese hogar.
Diferencia más pequeña que podemos medir	Con 32.993 hogares, mejoras menores a ≈ 0.0025 de F1 no se distinguen del ruido.

Glosario de apoyo; se muestra solo si lo preguntan.

Apéndice: hiperparámetros del LightGBM ganador

Hiperparámetro	Rango explorado	Elegido	Qué significa
Número de árboles	[200, 2000]	1.852	muchos árboles
Tasa de aprendizaje	$10^{[-3,\ -0{,}7]}$	0,011	pasos pequeños
Profundidad máxima	[3, 12]	12 (31 hojas)	árboles pequeños
Mínimo de hogares por hoja	[10, 200]	171	no memoriza casos
Variables por corte	[0,2; 1]	0,82	algo de azar
Hogares por árbol	[0,5; 1]	0,70	algo de azar
Ganancia mínima para cortar	$10^{[-10,\ 1]}$	7×10^{-10}	sin restricción

Búsqueda aleatoria (hipercubo latino) de 50 combinaciones, validación cruzada de 5 bloques sobre las 48 variables. Fuente: views/tablas/leaderboard_interno.csv (n6_lightgbm).

Apéndice: hiperparámetros del LightGBM ganador

Universidad de los Andes

Hiperparámetro	Rango explorado	Elegido	Qué significa
Número de árboles	[200, 2000]	1.852	muchos árboles
Tasa de aprendizaje	$10^{[-1.5,-0.5]}$	0,011	pasos pequeños
Profundidad máxima	[3, 12]	12 (31 hojas)	árboles pequeños
Mínimo de hogares por hoja	[10, 200]	171	no memoriza casos
Variables por corte	[0,2; 1]	0,82	algo de azar
Hogares por árbol	[0,5; 1]	0,70	algo de azar
Ganancia mínima para cortar	$10^{[-10,-6]}$	7×10^{-10}	sin restricción

Búsqueda aleatoria: 50 pruebas dentro de 50 combinaciones, validación cruzada de 5 bloques sobre los 20 mil hogares. Fuente: `views/tables/leaderboard_interno.csv` (`id2_lightgbm`).

Respaldo: la configuración que se reutilizó en todos los modelos con ingreso estimado.

Apéndice: en validación repetida, ninguna alternativa mejora de forma medible

Universidad de los Andes

Candidato	F1 validación repetida (EE)	Diferencia vs. LightGBM [IC 95 %]	F1 apartado [IC 95 %]
LightGBM base	0,7752 (0,0008)		0,7806 [0,7731; 0,7885]
+ redes (mezcla sin rediseño)	0,7758 (0,0008)	+0,0006 [−0,0001; +0,0013]	0,7820 [0,7743; 0,7897]
+ redes + rediseño (final)	**0,7759 (0,0008)**	+0,0007 [+0,0002; +0,0012]	0,7817 [0,7742; 0,7894]
CatBoost	0,7722 (0,0007)	−0,0030 [−0,0042; −0,0018]	0,7786 [0,7707; 0,7864]
Rediseño desde cero	0,7478 (0,0008)	−0,0274 [−0,0289; −0,0260]	0,7499 [0,7420; 0,7581]

15 evaluaciones (3 particiones nuevas × 5 bloques), IC pareados ingenuos. Con la corrección de Nadeau y Bengio el IC del modelo final es [−0,0004; +0,0017].
Fuente: views/tablas/nivel7/etapa16_fase1_resumen.csv, etapa18_resumen.csv.

Apéndice: en validación repetida, ninguna alternativa mejora de forma medible

Universidad de los Andes

Candidato	F1 validación repetida (SE)	Diferencia vs. LightGBM [IC 95 %]	F1 apartado [IC 95 %]
LightGBM base	0,7752 (0,0008)		0,7866 [0,7731; 0,7985]
+ redes (inactiva sin rediseño)	0,7750 (0,0008)	+0,0006 [−0,0001; +0,0013]	0,7868 [0,7743; 0,7987]
+ redes + rediseño (final)	**0,7759 (0,0008)**	+0,0007 [+0,0002; +0,0013]	0,7867 [0,7742; 0,7984]
CatBoost	0,7722 (0,0007)	−0,0030 [−0,0042; −0,0018]	0,7796 [0,7707; 0,7864]
Rediseño desde cero	0,7478 (0,0006)	−0,0274 [−0,0286; −0,0260]	0,7499 [0,7420; 0,7581]

Elaboración propia. Experimentos ejecutados en la Máquina 2 (Contenedor Progressus). Con la excepción de TabNet y Stacking (GPU de Kaggle Tesla P100, ~12,6 GB), todos los modelos fueron entrenados en CPU. Archivos fuente: `results/tables/cloud/Etapa2c3_Stack_resumen.csv`, `etapa2d_resumen.csv`.

Respaldo: CatBoost pierde de forma medible; la mezcla gana por milésimas y la ventaja no sobrevive a la corrección.